

Chapter 1 – Quick Start

Ready to start? We are going to dive right in, do a quick pass through the tool, and create a simple SysML V2 model of a doorbell system. Don't worry too much if the SysML V2 concepts presented in this chapter seem strange; we will come back to explain them in more detail in subsequent chapters.

State of SysML V2 and this Tool

Screen shots in this book were captured using a Windows 11 laptop configured to run *Cameo Systems Modeler version 2026x HF1*.

- The specific 2026x HF1 version of the tool is a released production version of the tool, not a beta.
- SysML V2 modeling is supported only in the enterprise edition of *Cameo Systems Modeler*.
- *OMG Systems Modeling Language™ (SysML®) Version 2.0* was publically released on September 29, 2025.

Neither the standard nor the tool are formally anything other than “released”. That having been said, the OMG working groups for the standard are still quite active and many small details are still being ironed out. For example, at the time of writing, the definition of tables is still being worked on in the Graphical Specifications working group. *Dassault Systèmes* is actively supporting these discussion. As such, we can expect ongoing updates to both the standard and the tool. Since we are writing and releasing this chapter very early in the development of the overall book, we will probably need to update this material one or more times before the final release of the book. The reader will notice that the downloadable “Quick Start” version of this chapter contains a book version number in the page footer that will be updated each time we release an update to this material.

Introducing the Tool

Before we start constructing our first modeling **project**, there are a few routine housekeeping matters to take care of.

Starting the Tool

Assuming that you selected the installation option to have *Cameo Systems Modeler* install a Windows desktop icon, the easiest way to start the tool is to click on that icon. ⁽¹⁾

⁽¹⁾ *Dassault Systèmes* releases the tool under several different brand names. In this book we are presenting *Cameo Systems Modeler version 2026x Enterprise Edition*, version 2026x HF1. However, other than perhaps the splash screen, all the different tool versions should be identical for the material being covered in this book. Note, however, where “Enterprise” and “Architect” versions are offered, only the “Enterprise” supports SysML V2 modeling.

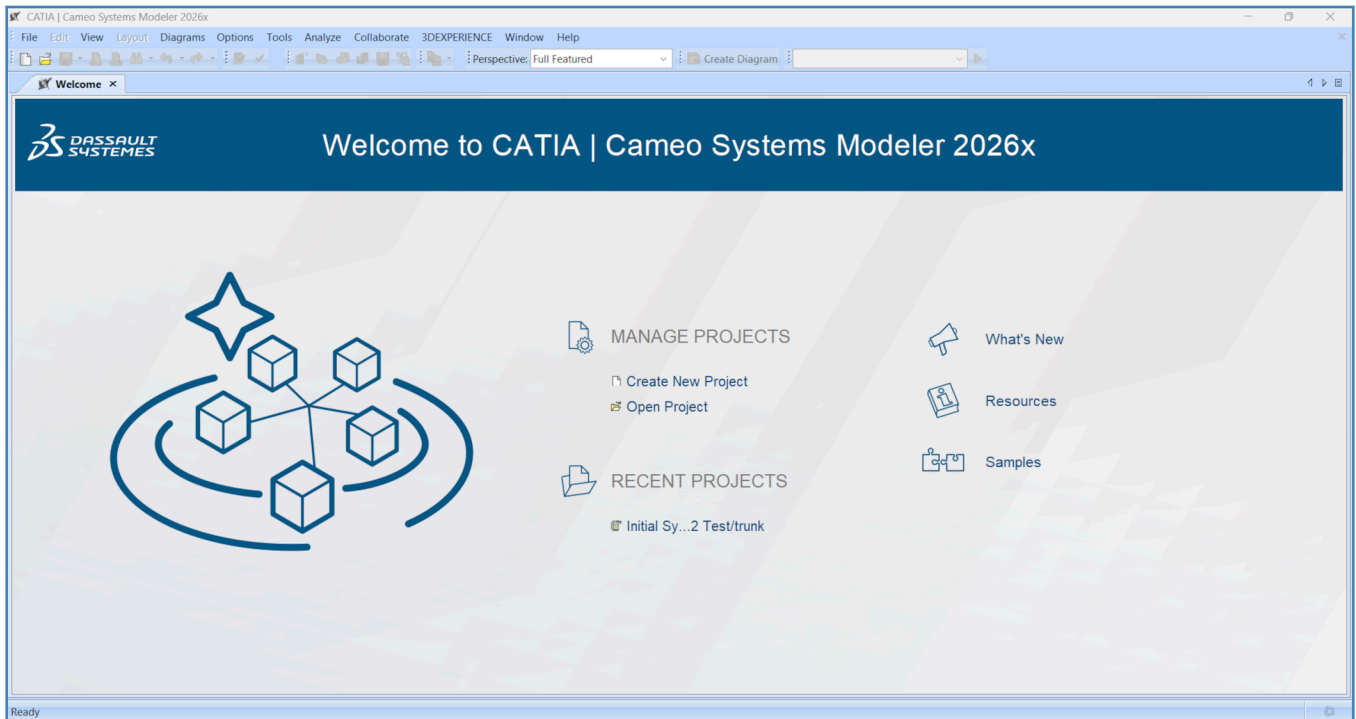


Figure 1-1 – Start page

This start page presents the standard “File, Open” interface.

Managing Modeling Projects

Starting with version 2026x, *Cameo Systems Modeler* manages local projects in two different modes.

- **SysML V1 Projects** – Projects containing SysML V1 models are stored in the familiar local “*.mdzip” file format.
- **SysML V2 Projects** – Projects containing SysML V2 models are stored in the new **local repository**.⁽²⁾

The local repository can be thought of as a sort of local, built-in instance of the cloud server for collaboration called *Teamwork Cloud*.⁽³⁾ that *Dassault Systèmes* sells for model version control and team collaboration.

The local repository contains features for keeping track of versions of your modeling projects. A modeling project can also be exported in two formats:

⁽²⁾ The local repository is supported only in the enterprise edition of the product to support SysML V2 modeling. The local repository is not visible in the architect edition of the product.

⁽³⁾ See: <https://www.3ds.com/products/catia/no-magic/teamwork-cloud>

- ***.mdszip** – This format is a zip file that contains the tool internal representation of the model, diagrams, and other information. This is not an interchange format.
- ***.sysml** – This format is the SysML textual notation for the model. This format is described in the KerML specification in section 10.2 *Model Interchange Formats*.

Creating a Modeling Project

Now we are ready to create our first modeling **project**. In *Importing a Modeling Project* on page 34, we will discuss the mechanics of loading a modeling project from the example files provided with this book. For the moment, we will concentrate on creating a new modeling project from scratch.

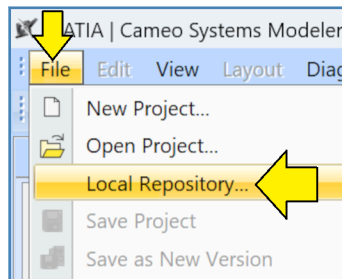


Figure 1-2 – Creating a new project

Pull down the menu in the top left corner of the screen and select “Local Repository...” (not “New Project...”) to start a new project.

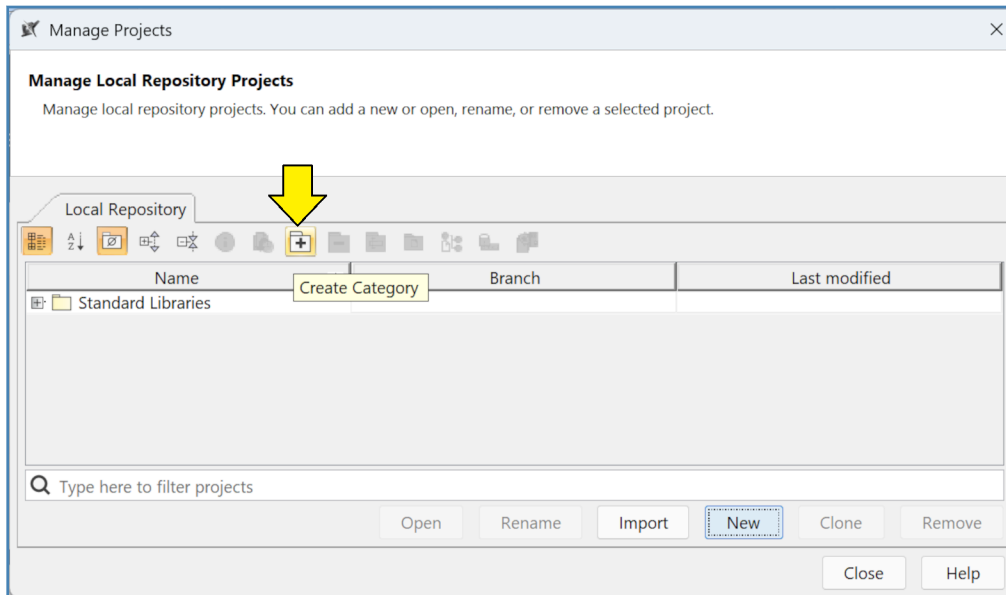


Figure 1-3 – Create a category

The local repository will appear. The first thing you will want to do is create a category for your modeling projects. Click on the folder icon that contains a “+” sign.

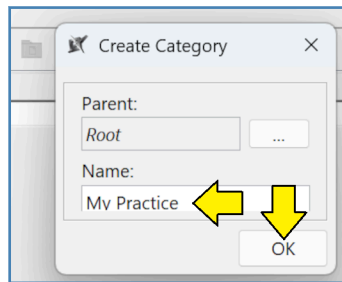


Figure 1-4 – Name the category

Name the category. You can name the category anything that makes sense for you. The tool allows for multiple categories and categories can be renamed. For the moment, we will name the category “My Practice”. Enter the category name and click “OK”.

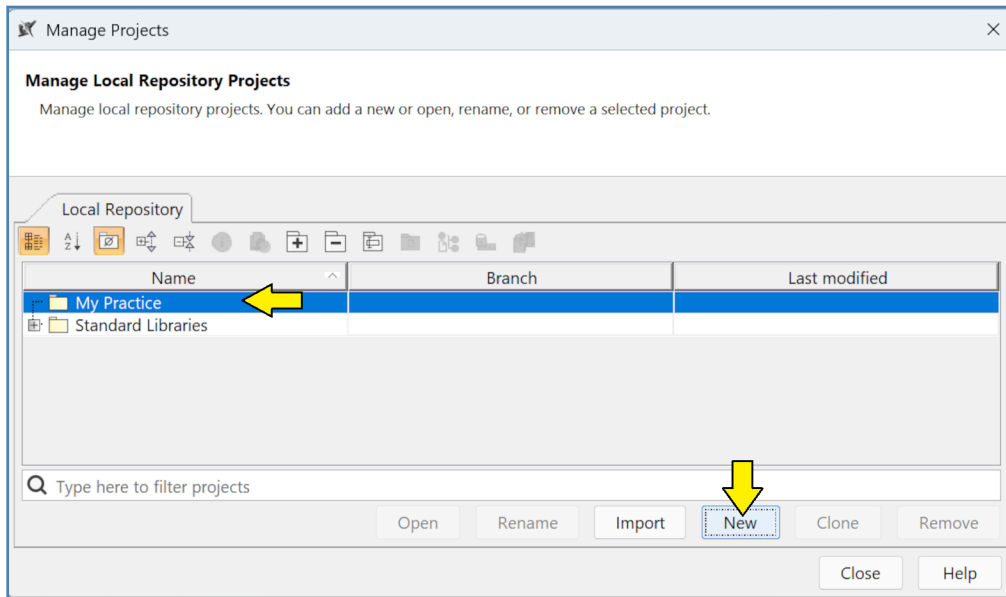


Figure 1-5 – Create a project in the category

The new category is now visible in the local repository. Click “New” to create a project. (The local repository should be displayed with the new category selected, but it is not crucial that the category be selected at this step as we will get an opportunity to change the category in the next step.)

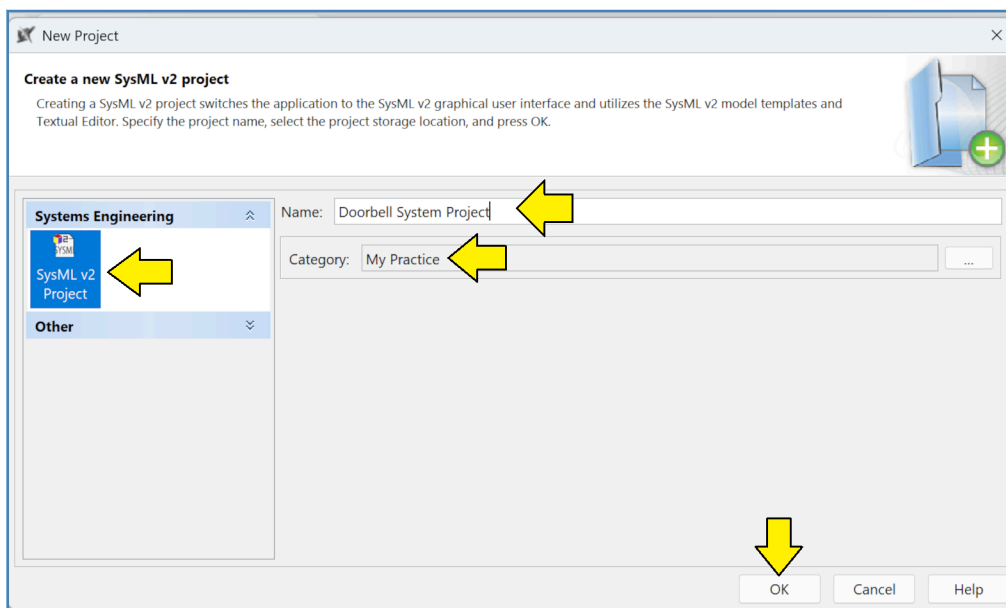


Figure 1-6 – Name the new project

- 1) As of this writing, the local repository only supports SysML V2 projects. In later releases, other project types might be added.
- 2) Check that the proper category is selected.

- 3) Give the new project a name. Our first example will be a doorbell system. We recommend that you name the project “Doorbell System Project”.
- 4) Click “OK” to create the new model.

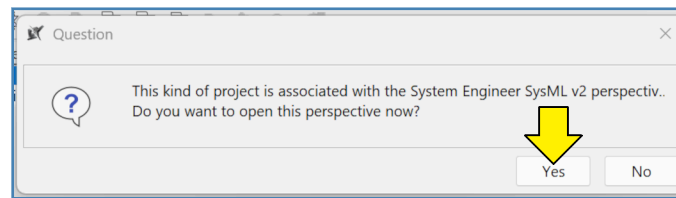


Figure 1-7 – Accept system engineer perspective

Click “Yes” to accept the system engineer perspective. ⁽⁴⁾

A progress panel will appear, followed by a productivity tip panel. After some time, the empty modeling project will appear.

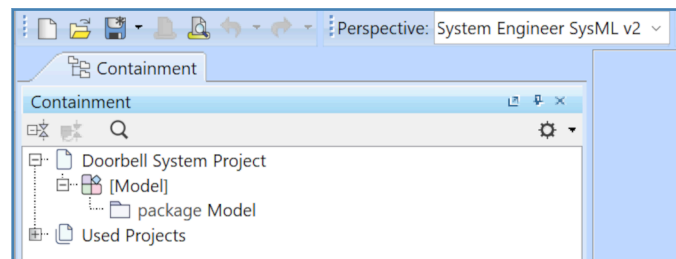


Figure 1-8 – Empty project

We now have a sort of “empty” modeling project. Actually, it isn't really empty. If we take a look at the tree structure at the left of Figure 1-8, we can see several things.

- 1) **The project** – This is the top of the project. One project can contain multiple models and libraries.
- 2) **A namespace** – We don't need to worry too much about this thing at this point. A project can contain multiple namespaces. We will explain more about namespaces later in the book.
- 3) **A package** – The tool has created an initial **package** for us. As we will see shortly, packages are our main tool for organizing the content of our model. Notice that the tool has given us a hint by naming the default package “model” that this is the destination for our SysML V2 modeling content.
- 4) **Used projects** – This is where the tool shows us that it has attached a subset of the SysML V2 and KerML standard libraries to our project.

⁽⁴⁾ While creating SysML V2 modeling projects from the local repository, there really is not much choice here. In the future, other types of modeling projects may be supported and these might have different perspectives associated with their modeling language types.

We will be covering projects, namespaces, models, libraries, and usages in more detail later in the book. For the moment, we can focus on the package that will contain our SysML V2 model content.

Creating a View and a Simple System

We will start the construction of our model with the **package**. Packages are a central feature of SysML V2 models. You can think of packages as being very similar to directories in the Windows file system. You can name packages freely. You can put packages within packages. While it is possible to put elements right at the root of the model, your model will get cluttered and unreadable very quickly if you try to model that way.

Conveniently, the tool has created a first package for us called “Model”. Let's make that a little more specific by renaming the package to “Doorbell System Model”.

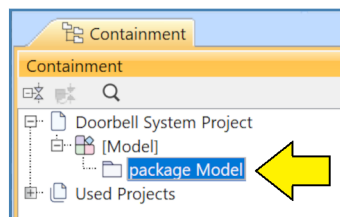


Figure 1-9 – Select the default package

With the default package selected, press F2 and rename the package.

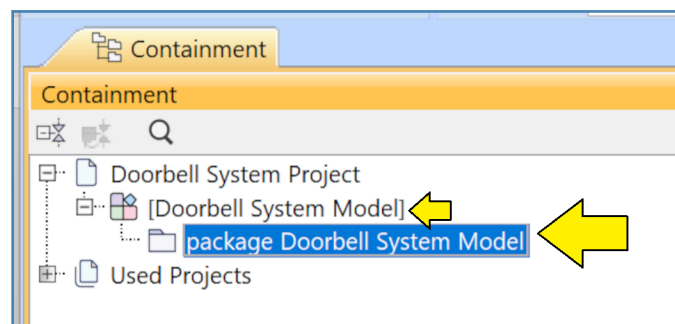


Figure 1-10 – Rename the default package

Notice that the tool automatically renames the namespace to match the renamed package.

Now we are ready to add our first view. Note that SysML V2 refers to diagrams as **views**.

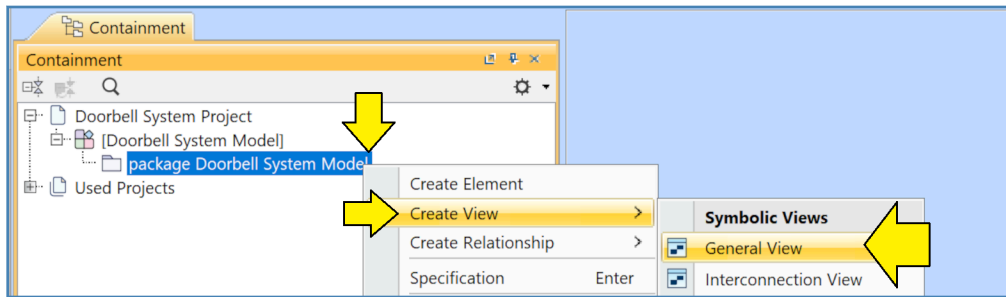


Figure 1-11 – Create a view

With the package still selected, right-click and select “Create View” followed by “General View”.

Name the view “Doorbell System Structure Overview”.

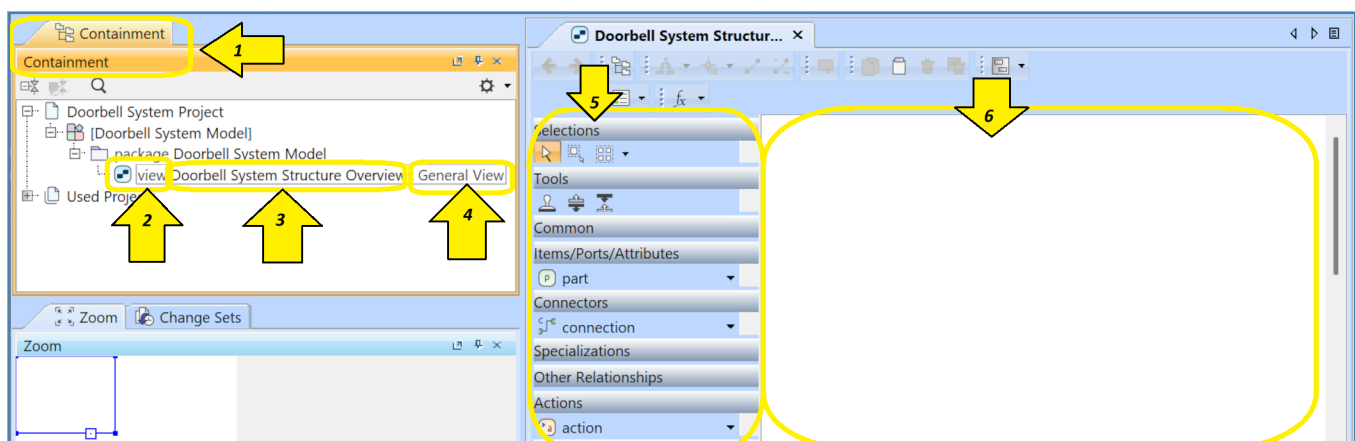


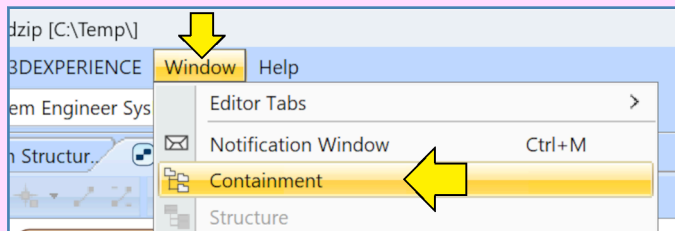
Figure 1-12 – Tool layout overview

Now, with the first view created, we can take a closer look at the tool layout.

- 1) The panel at the top left of the tool is the **containment tree**. This presents a familiar browser list of elements of the model. The plus and minus symbols can be used to expand or collapse branches of the tree.
- 2) The tool has created as a **view** element. Views are themselves formal SysML V2 model elements.
- 3) The name of the view is shown next.
- 4) The view type of “General View” is shown next.
- 5) Within the view, the **diagram palette** contains tools that we will begin using next to model our system.
- 6) Finally, the view contains a blank area for us to begin modeling our system in.

What if the Containment Tab Isn't There?

What if the containment tree isn't visible on your screen? This kind of thing happens all the time with this sort of highly configurable tool.



The containment tree can be displayed by selecting “Window” and then selecting “Containment”.

We are now ready to create our first **part**.

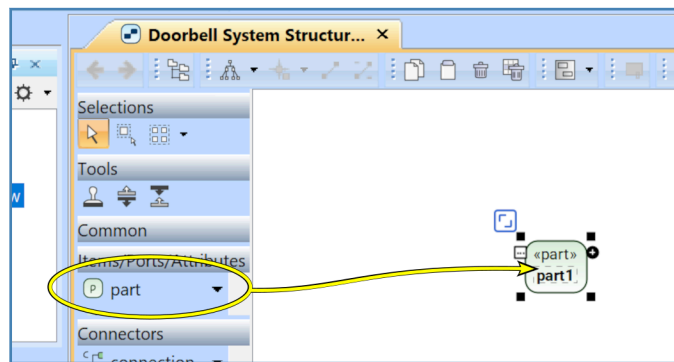


Figure 1-13 – Create a part

In the diagram palette, click on the “part” tool and drag it to the view.

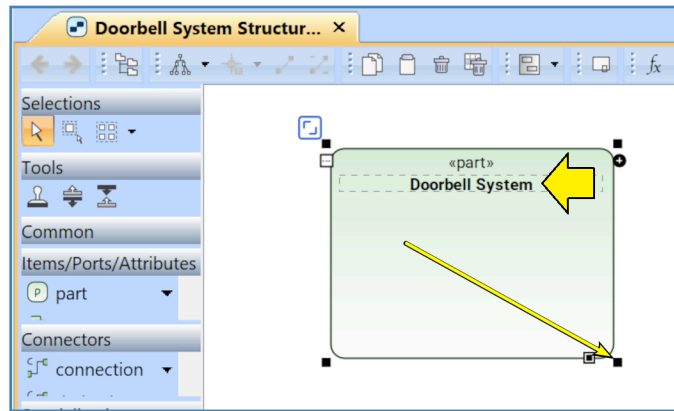


Figure 1-14 – Rename and expand the part

Rename the part to “Doorbell System”. Drag the lower-right corner of the shape downwards and to the right to expand the shape.

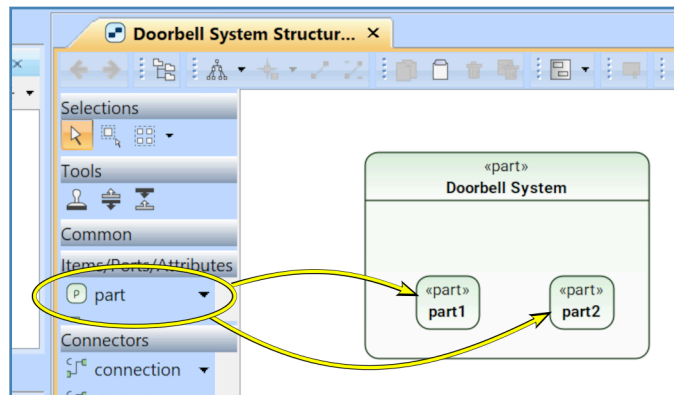


Figure 1-15 – Add two nested parts

Use the part tool again to add two nested parts within the “Doorbell System” part.

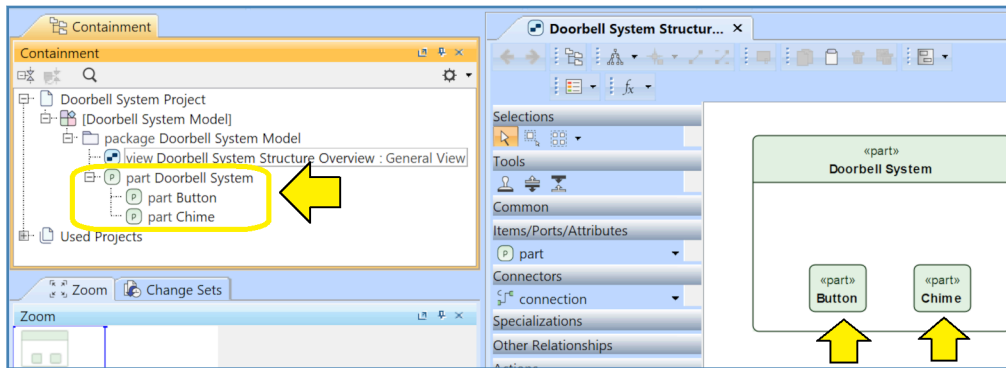


Figure 1-16 – Rename nested parts

Rename the nested parts “Button” and “Chime”. Notice that the new parts are nested within “Doorbell System” in the view and also in the containment tree.

Our First Glimpse of Model-Based Systems Engineering (MBSE)

We are at now at the starting line for “Model-Based Systems Engineering (MBSE)”. That is, the containment tree is showing “the model” and the view is just a view. We could make two or three more views with the same “Doorbell System” part. In the model there would still be one and only one part. That is, if we think of the model as a building, the views are like snapshots taken of the building from different angles. The snapshots (views) are not the building. The model is the building.

A model element can be shown in as many or as few views as desired. In fact, you can create a model element directly in the containment tree and not include it in any views at all. This flexibility to make many different views including the same element becomes a central feature of MBSE because this technique allows the systems engineer to make many simple views for different stakeholders, each view showing only the model elements of interest to that stakeholder.

A Quick Look at the Text Syntax

One of the key differences between SysML V1 and SysML V2 is the unambiguous text syntax for models. In *Exporting the Model SysML V2 Text Syntax* on page 32, we will discuss the mechanics of exporting the SysML V2 syntax. For the moment, we will skip that step and take a quick look at the text syntax itself.

```
package 'Doorbell System Model' {  
  view 'Doorbell System Structure Overview' : DS_Views::SymbolicViews::gv;  
  part 'Doorbell System' {  
    part Button;  
    part Chime;  
  }  
}
```

Figure 1-17 – Text syntax for programmers

Programmers and software engineers with extensive experience coding Java, C, or C++ will take a glance at Figure 1-17, yawn, and mumble something like: “It is so obvious.” If you are one of these software-intensive people you can skip the rest of this section.

For other non-software readers ⁽⁵⁾, we will take this little snippet of SysML V2 text syntax and expand it a little bit. ⁽⁶⁾

```
package 'Doorbell System Model' {  
  view 'Doorbell System Structure Overview' : DS_Views::SymbolicViews::gv;  
  part 'Doorbell System' {  
    part Button;  
    part Chime;  
  }  
}
```

Figure 1-18 – The boundaries of the package

The first thing to understand is that the text syntax is a series of nested elements, similar to Fabergé eggs. In Figure 1-18, we can see that:

- The name of the package is highlighted in yellow. Notice that the name of the package is enclosed in single quotation marks. The quotation marks allow the name to include spaces. In fact, it allows the name to include almost all Unicode characters, leaving you as a modeler free to name model elements in Japanese, Thai, Arabic, Lithuanian, or pretty much any human language that can be shown in a UTF-8 encoded website.
- Everything inside the package is highlighted in pink. Notice that the list starts with a “{” character and ends with a “}” character.

⁽⁵⁾ and perhaps for “software” readers who have never coded in anything other than Python...

⁽⁶⁾ Notice that in Figure 1-17 is that the tool has exported the namespace containing the model, not the entire project. This behavior is in compliance with the KerML specification and we will cover this in more detail later in the book.

Like most computer programming languages (except Python) the SysML V2 text syntax is **white space invariant**. The style of placement of the open and close brackets and the content in between is for the convenience of the human reader and does not affect the meaning of the content. From the computer's point of view, this example of text syntax could all be in one long line.

```
package 'Doorbell System Model' {  
  view 'Doorbell System Structure Overview' : DS_Views::SymbolicViews::gv;  
  part 'Doorbell System' {  
    part Button;  
    part Chime;  
  }  
}
```

Figure 1-19 – Contents of the package

Finally, in Figure 1-19, we can see that:

- The package contains the view. Notice that the declaration of the view does not contain any curly brackets and instead ends with a semicolon. Again, this is common software syntax: one declaration either ends with a semicolon or expands into nested declarations with curly brackets.
- The package also contains a part declaration that itself includes two more part declarations.

For the rest of this chapter, we will not be looking further at the SysML V2 text syntax. However, we will be coming back to the topic in subsequent chapters. ⁽⁷⁾

Add an Action Flow

One of the most important (and often neglected!) steps in designing a solid system, is to sit down with the stakeholders and have a free form discussion about what they expect the system to do and how they expect the system to do it. In most cases, the stakeholders are not engineers and showing them a bunch of complicated diagrams or pages of software source code is counter-productive. In fact, it is usually better not to show them anything at all, at least initially. Get the discussion going. Make them tell you a story. Work with them to reformulate and polish the story until the story makes sense and they are enthusiastic about how the story flows.

For our doorbell system, our discussion with the stakeholders has yielded the following user story for our doorbell system:

⁽⁷⁾ At the time we are writing this book, the syntax for views is still evolving. By the time you are reading this section, the text syntax for the view produced by your tool may look slightly different.

“ The visitor walks up to the door of the house. The visitor finds that the doorbell button is glowing dimly. The visitor pushes the doorbell button. Immediately, a chime begins to sound. The doorbell button also begins to flash on and off. After five seconds, the chime stops sounding and the doorbell button returns to a dim glow. ”

Great! Now we have a user story. Let's turn that into an action flow. First, let's add a few packages to the model to keep things neat and organized.

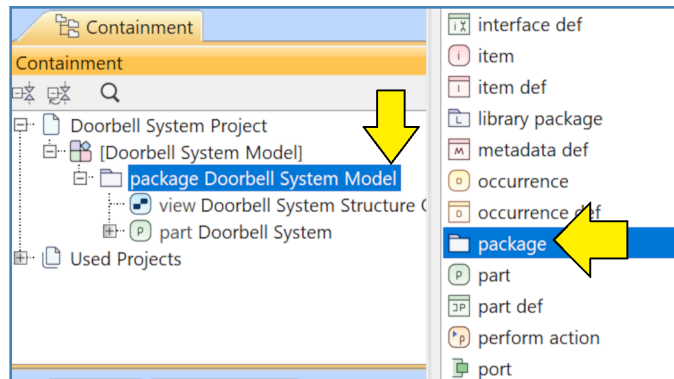


Figure 1-20 – Add another package to the model

Right-click on the system model package, select “Create Element” and then “package”. Name the new package “System Structure”.

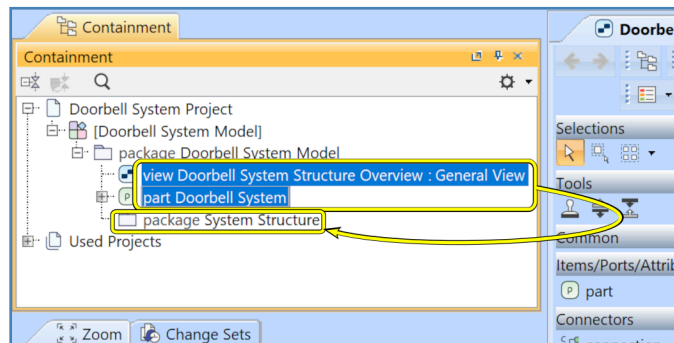


Figure 1-21 – Drag the parts and the view into the new package

Select the parts and the view and drag them into the new package.

Add an additional package to the system model package. Name the new package “System Behavior”.

A Word About Packages

If you have ever read or watched an advertisement about a delighted person who purchased an expensive “custom closet” service and was now free from the tyranny of the cluttered and over-stuffed closet, you will understand the basic role of packages in the model. Packages are like the beautifully organized dividers, drawers, and cubby holes in the custom closet. However, unlike the custom closet case, we do not need to pay for an expensive consulting service to rearrange the structure. We can create new cubby holes, rename old cubby holes, combine them, delete them, or move them around on the fly. For this reason, it is not immediately necessary to over-think the name of the package we are creating for our visitor arrival user story.

In order to model the user story, we will want to add an **action flow view** to the “System Behavior” package.

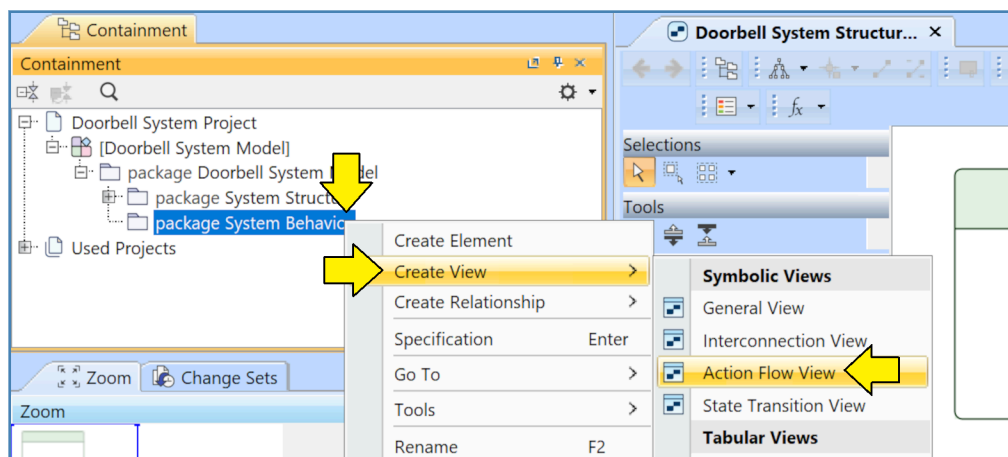


Figure 1-22 – Add an action flow view

Next, right-click on the “System Behavior” package, click “Create View”, and add an “Action Flow View”.

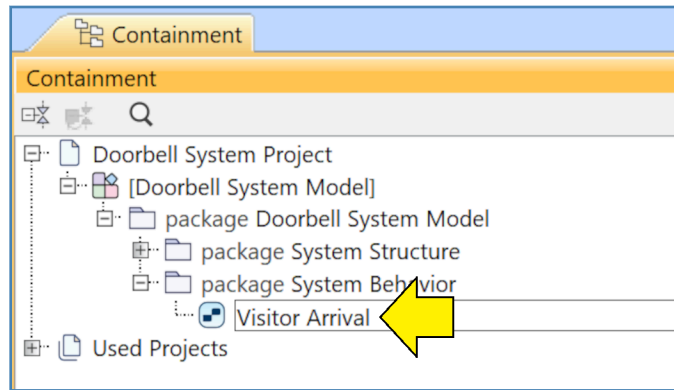


Figure 1-23 – Name the view

Name the new view “Visitor Arrival”.

Action Flow versus SysML V1 Activity Diagram

Readers with previous SysML V1 experience will notice a subtle difference at this point. The tool did NOT create an “activity element” and place the diagram within the element. The tool only created the view.

Does that mean that we don't need an enclosing element for the action flow? No, not quite. You will quickly notice that you can't do anything practical such as adding a **start** element without having an enclosing **action** to put it in. However, what has changed is that there is not necessarily a one-to-one correlation between an action flow view and a single enclosing element.

For readers without SysML V1 experience, just ignore the gibble in the previous few sentences and continue following the instructions below.

Next, we will want to add an **action** to the model to contain the user story.

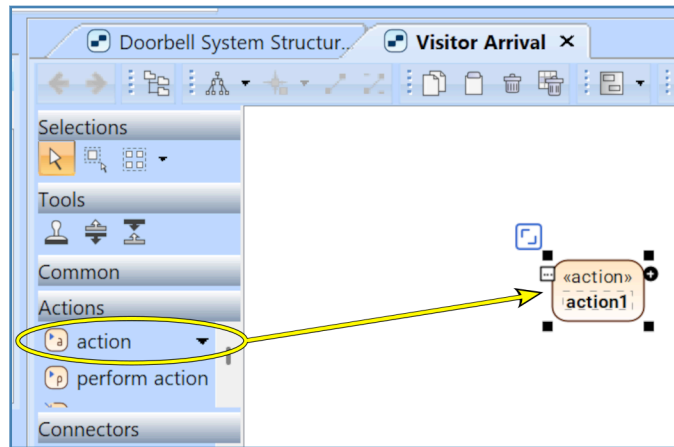


Figure 1-24 – Add an action to enclose the user story

In the diagram palette select the “action” tool and drag it to the view.

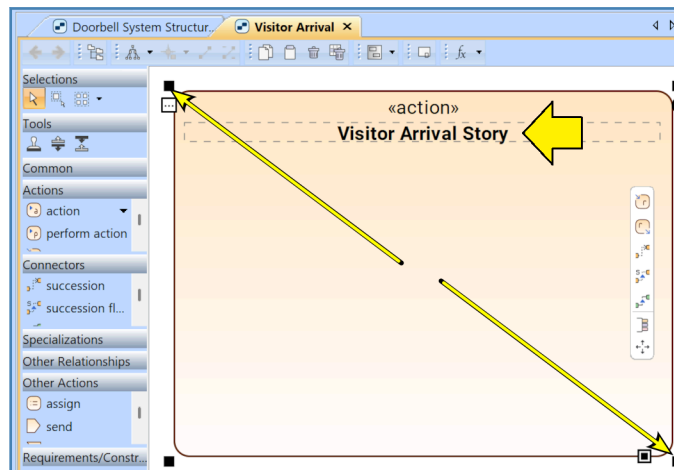


Figure 1-25 – Rename the action and drag to enlarge

Rename the action “Visitor Arrival Story” and drag the corners to make the shape larger.

Next, every story has to start somewhere, so we will add a **start** element to our model.

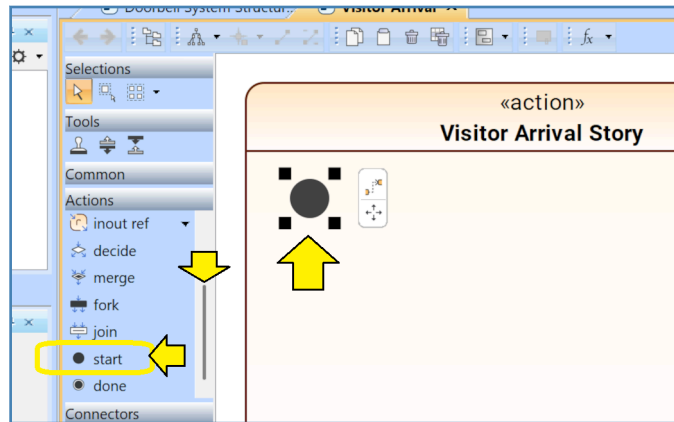


Figure 1-26 – Add a start element to the view

Find the “start” tool in the diagram palette. Depending on the size of your screen, you may need to use the scrollbar in the diagram palette to find the tool. You may need to collapse some of the other sections such as “Common” to make more room to scroll. Select the “start” tool and click in the view to create the start element. ⁽⁸⁾

Now we are ready to read through the user story and find things that happen (in other words, verbs) to model as actions. Returning to the user story, the first sentence is:

“The visitor walks up to the door of the house.”

For the moment, we want to focus on what happened, not who did it. We will model an **action**: “Walk up to door”. ⁽⁹⁾

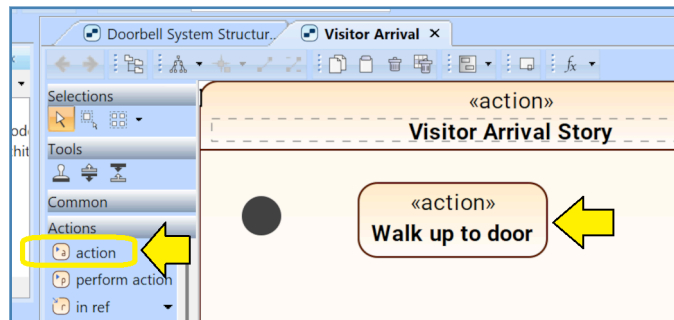


Figure 1-27 – Add the first action to the view

Find the “action” tool in the diagram palette. Select the tool and click to create an action in the view. Click inside the shape and rename the new action: “Walk up to door”.

⁽⁸⁾ Notice that this time we used a select-and-click operation rather than a drag-and-drop operation. This is a matter of personal preference. Most operations in the tool will work either way.

⁽⁹⁾ In naming actions, we usually dispense with punctuation such as periods.

Now that we have a start and an action, we need to connect them. In SysML V2 this is done with a **succession**.⁽¹⁰⁾

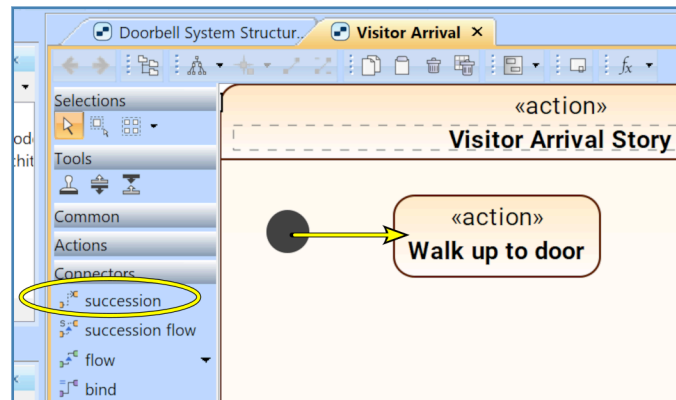


Figure 1-28 – Add the first succession to the view

Open the “Connectors” group in the diagram palette and select the “succession” tool. In the view, click on the start element and drag an arrow to the new action.⁽¹¹⁾

Next, let's return to the next sentence in the user story:

“The visitor finds that the doorbell button is glowing dimly.”

Hmmm. This is one sentence, but two different things are happening. The button is glowing dimly, and the visitor has noticed the glow.⁽¹²⁾ We will model these as separate actions – again, for the moment we will not worry about who does what.

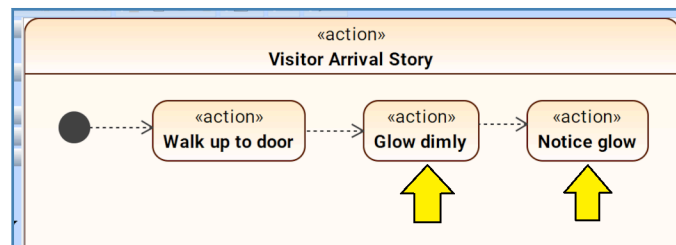


Figure 1-29 – Add two more actions

Add two more actions and the appropriate successions called “Glow dimly” and “Notice glow”.

⁽¹⁰⁾ Veterans of SysML V1 will notice here that the SysML V2 “succession” replaces the SysML V1 “control flow”.

⁽¹¹⁾ You can also select, click on the start, and then click on the action.

⁽¹²⁾ At this point, the exasperated engineer asks: “ Why can't the stakeholders be more clear and state what they want in nice, atomic sentences!?” The answer is that this sort of engineering formulation is not the stakeholders' job. It is our job as systems engineers to work with the stakeholder input and sort out small details like this as we go.

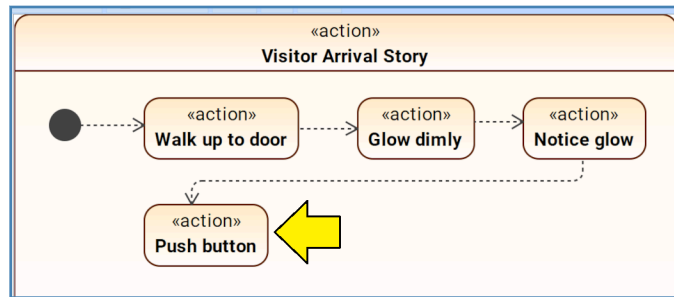


Figure 1-30 – Add action to represent pushing the button

Add an action to represent pushing the button.

The next two sentences in the user story are interesting.

“Immediately, a chime begins to sound. The doorbell button also begins to flash on and off.”

Two different things are happening in parallel. SysML V2 has a pair of concepts to model such things: **fork** and **join**. Let's add a fork to the model.

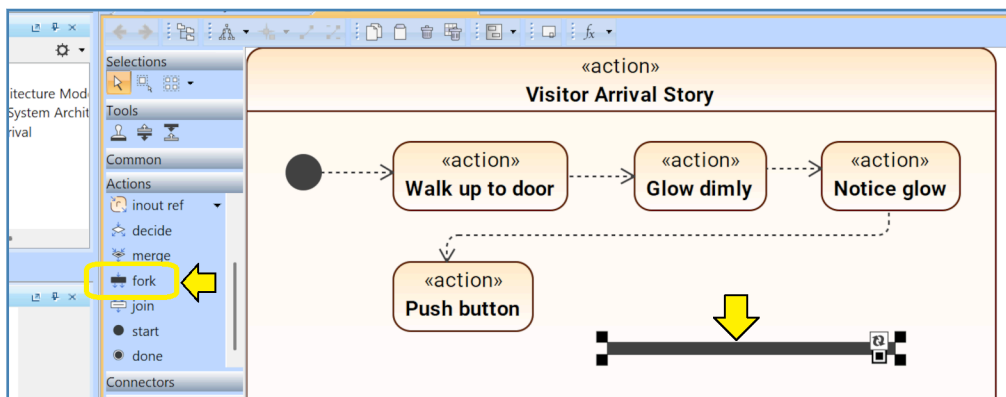


Figure 1-31 – Add a fork

In the diagram palette, select the “fork” tool and add a narrow, horizontal bar to the view.

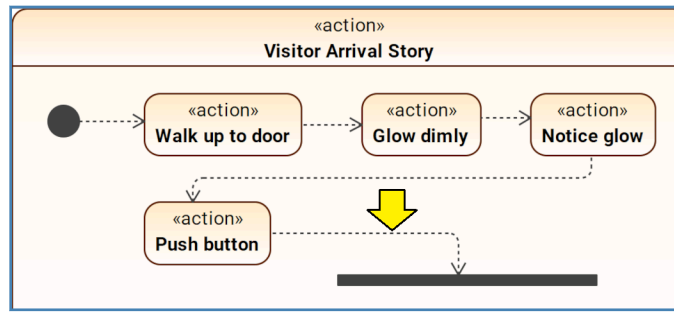


Figure 1-32 – Connect the fork

Use the succession tool to connect the fork. The black horizontal bar (the fork) indicates that the sequence will split and proceed down two or more paths in parallel. Next, let's add the two paths.

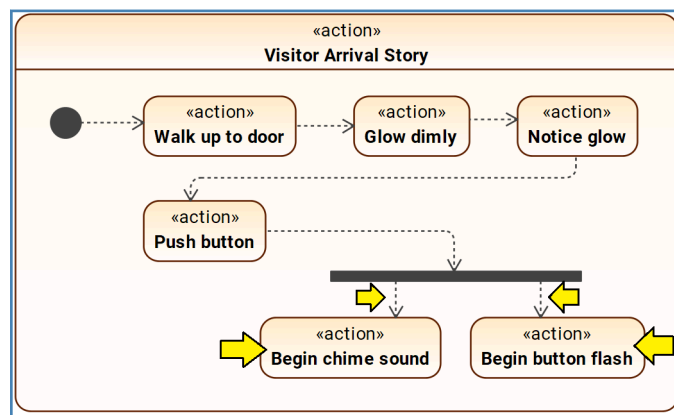


Figure 1-33 – Add flash and chime actions

Below the black horizontal bar, add an action for “Begin chime sound” and another for “Begin button flash”. Use the succession tool to connect from the fork to each action.

We will need some more space to continue the dual threads. Drag the corner of the enclosing action downward to make space.

Returning to our user story, the next (and final) sentence is:

“After five seconds, the chime stops sounding and the doorbell button returns to a dim glow.”

First, we need to think about the five second timeout. There are actually several ways we could model such a timeout in a more complete model. For the moment, we will model it here as a loop. Many readers who have done at least a little bit of script programming or have some familiarity with flow charts from other places will recognize the “do while” pattern. The key feature here is a **decide** element to determine whether five seconds have passed yet or not. We will need one for each thread.

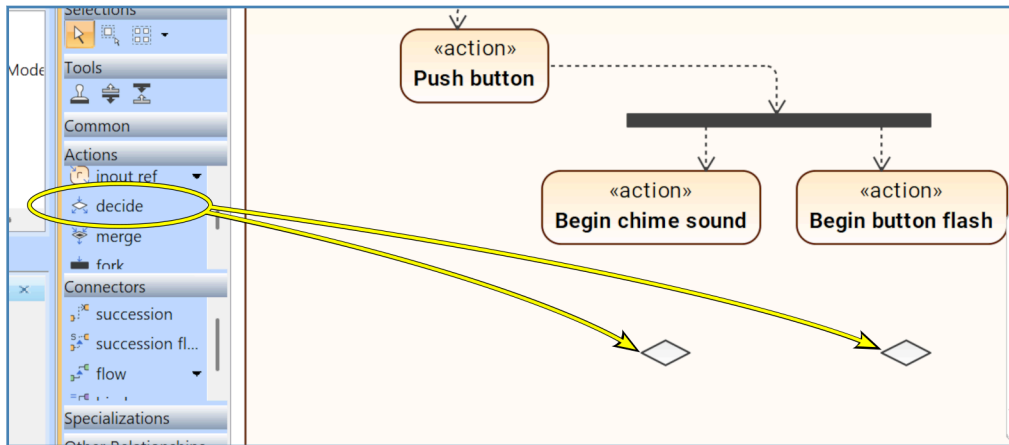


Figure 1-34 – Add two decide elements

Use the “decide” tool in the diagram palette to add two decide elements to the view. These two diamonds do look a bit like the diamond shapes in a classical flowchart, but there are a few more steps that are unique to SysML V2. The first is that we can't just loop back to a random point in the previous sequence. We need to provide a **merge** node to connect to.

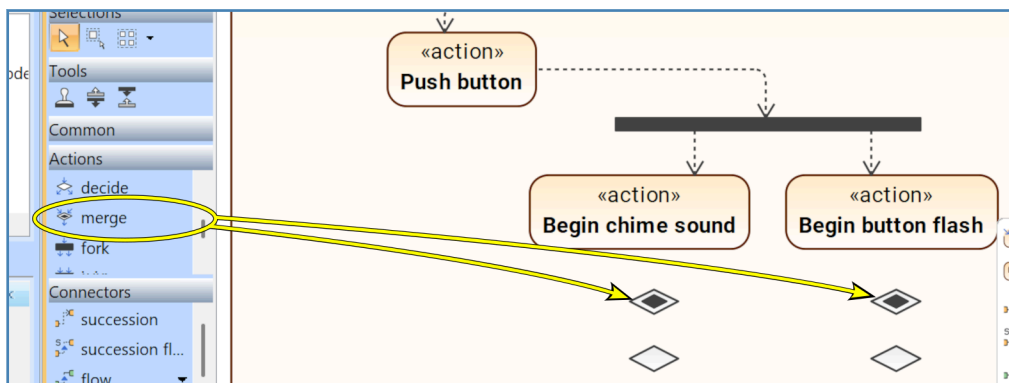


Figure 1-35 – Add two merge elements

Use the “merge” tool in the diagram palette to add two merge elements to the view above the decide elements. Notice that the merge elements look similar to the decide elements, but with a solid black diamond in the interior.

Next, let's add the actions that we want to occur after the five second timeout has elapsed.

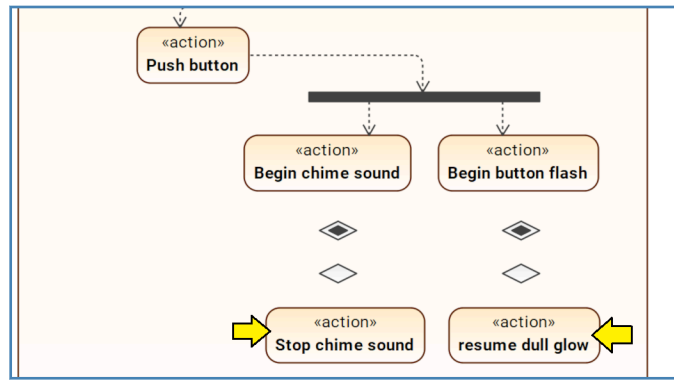


Figure 1-36 – Add actions to stop the flash and chime

Add the actions to stop the chime sound and resume a dull glow. Now we are ready to wire it up!

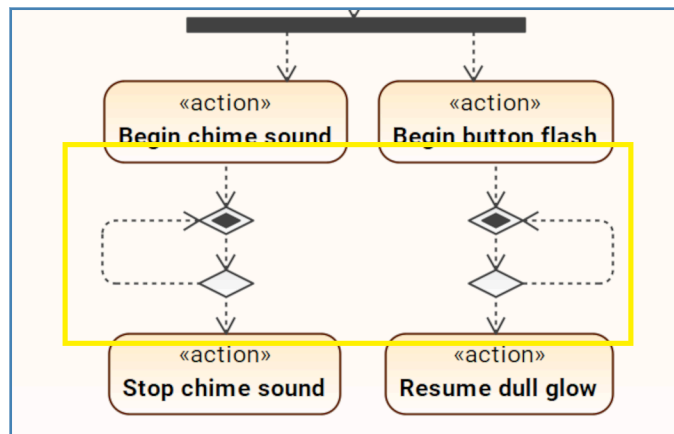


Figure 1-37 – Add successions

Use the succession tool in the diagram palette to wire up the decide and merge elements as shown in Figure 1-37.

Guards

What we would want to do next is add **guard** conditions on the successions coming out of the decisions about such as “time ≤ 5s” for the successions leading back to the merge nodes and “time > 5s” for the successions continuing on to stop the chime or button flashing. Unfortunately, the current test version of *Cameo Systems Modeler* does not yet support this step from the graphical user interface. Once this function is provided by *Cameo Systems Modeler*, we will update this section to show the steps to add these guards. In the meantime, we will mock up the view with text boxes from “Common” section of the diagram palette to look like guards.

In SysML V2, **guards** are both more semantically precise and less conveniently flexible than guards in SysML V2. Guards have to be a clear Boolean value. At this point, it is not entirely clear how much flexibility there

will be in the implementation. The current trend is towards declaring a Boolean attribute and explicitly testing that to see whether it is true or false. In any event, this functionality is not quite working yet. Instead, we will visually mock it up in the view using text boxes.

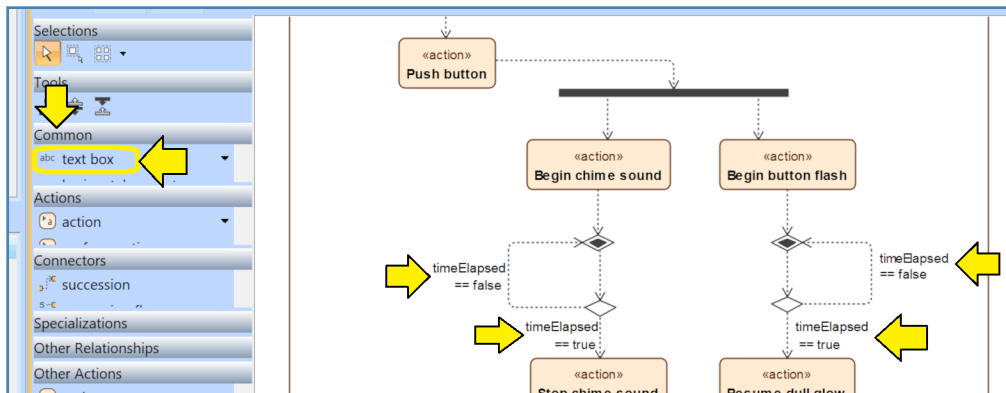


Figure 1-38 – Mock up the guards with text boxes

- 1) Imagine that we have added a Boolean property “timeElapsed” to the activity (not shown).
- 2) For the moment, we assume that some other process is updating the “timeElapsed” property.
- 3) In the “Common” section of the diagram palette find the “text box tool”.
- 4) Add text boxes to the two successions that lead back to the merge nodes containing the text: “timeElapsed == false”.
- 5) Add text boxes to the two successions that lead forward to stop the chime sound and resume dull glow containing the text: “timeElapsed == true”.

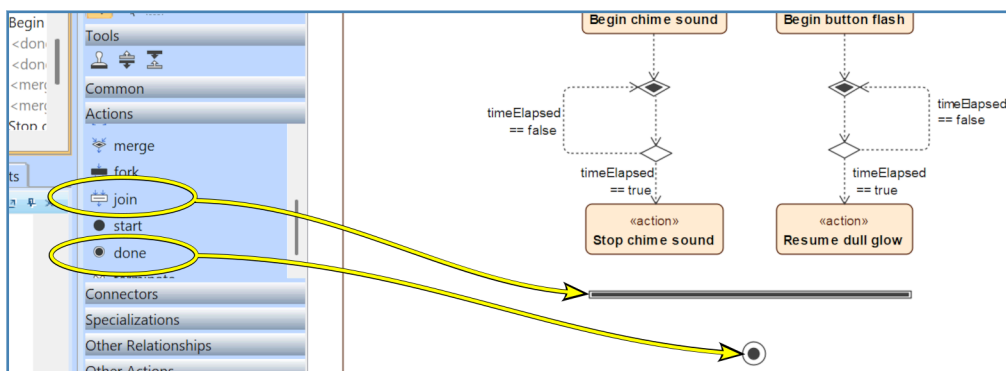


Figure 1-39 – Add join and done

In the diagram palette, find the **join** and **done** tools and add elements to the view as shown.

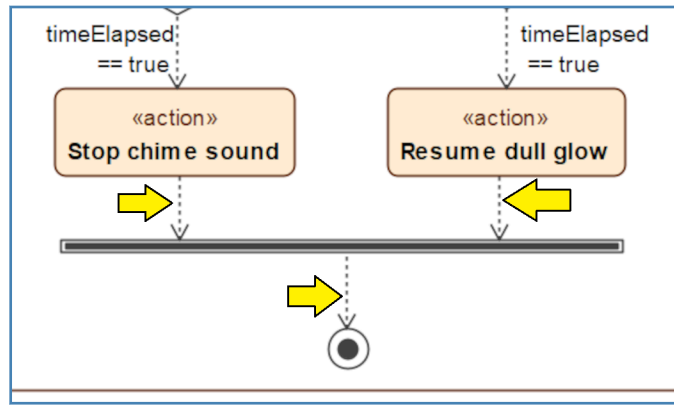


Figure 1-40 – Add successions to finish view

Add successions to finish the view as shown.

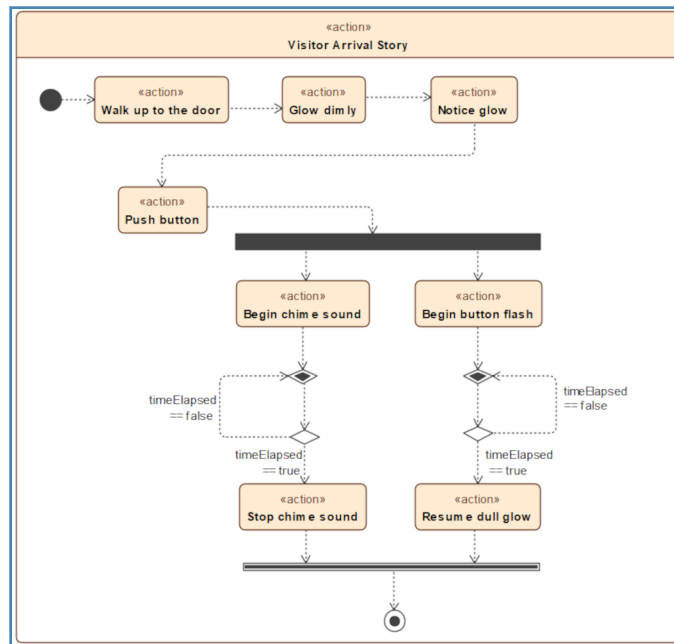


Figure 1-41 – Completed view

Add Some Requirements

In the examples, you will find a spreadsheet called “Doorbell-System-Requirements.xlsx” that contains some requirements for our doorbell system.

ID	Name	Description
CH-01	Chime Tone	The chime tone shall consist of one second of higher pitch bell tone, followed by one second of lower pitch bell tone, followed by one second of silence.
CH-02	Chime Cycle	The chime cycle shall consist of three chime tones.
CH-03	Chime Cycle After Button Press	When the doorbell button is pressed, the chime unit shall complete one chime cycle.
BT-01	Flash During Chime	While the chime cycle is in progress, the doorbell button shall flash.
BT-02	Flash Cycle	The button flash cycle shall be 0.5 second bright illumination followed by 0.5 second of no illumination.
BT-03	Idle Dim Glow	When the doorbell system is idle, the doorbell button shall glow dimly.

Figure 1-42 – Legacy requirements for the doorbell system

Apparently, someone extracted this spreadsheet from our company legacy requirement management system. Sharp-eyed readers will notice that these legacy requirements don't completely perfectly match the user story provided by our current stakeholders. This sort of requirements-reality mismatch is a common problem with requirements extracted from legacy requirements management systems. Obviously, we would want to review the requirements with our stakeholders and update the requirements to correct any discrepancies. For the moment, however, we are not going to worry about the details and instead focus on the mechanics of modeling the requirements in SysML V2 using *Cameo Systems Modeler*.

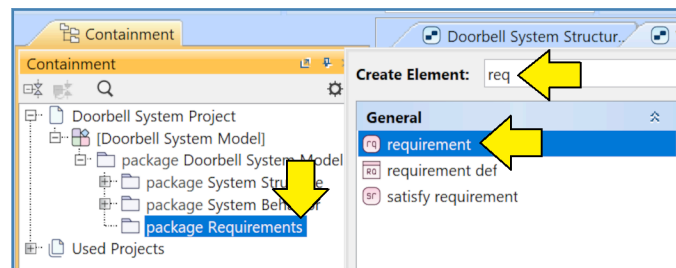


Figure 1-43 – Add package and create first requirement

Add a new package called “Requirements” to the model. Right-Click on the new package and select “Create Element”. In the pop-up panel, enter “req” in the create element search field. The list will shrink. Select “requirement”.

We will model the second requirement “CH-02 Chime Cycle”.

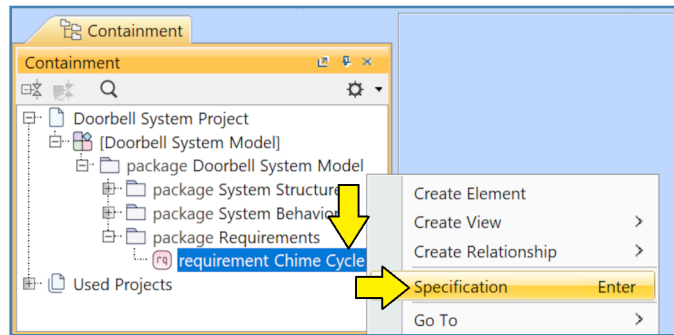


Figure 1-44 – Rename the requirement and open its specification

Rename the requirement “Chime Cycle” in the containment tree. Right-click on the requirement and select “Specification”.

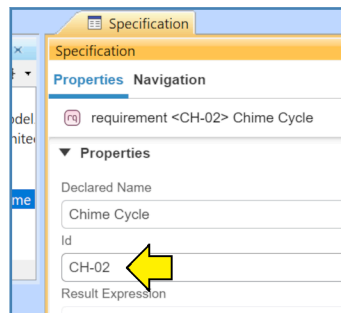


Figure 1-45 – Enter the Id

Enter the Id “CH-02” in the field provided. ⁽¹³⁾

⁽¹³⁾ Sharp-eyed readers will notice that the Id has been assigned to the **short name** for the SysML element. Overloading the requirement Id onto the SysML V2 short name is just one of the odd decisions made by the SysML V2 team. Next, we will see that the SysML V2 team decided to overload the requirement text onto the documentation field, a likewise odd decision.

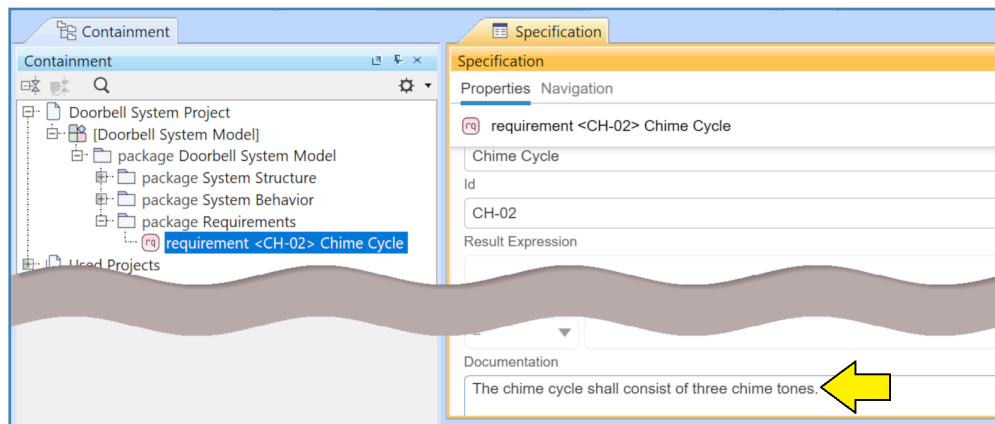


Figure 1-46 – Enter the text of the requirement

Enter the text of the requirement in the documentation field. Click the “x” in the upper right-hand corner of the specification window to close it.

We can now create another general view in the requirements package and name it “Requirements Traceability”. If you are new to systems engineering, don't worry about the word “Traceability” for the moment.

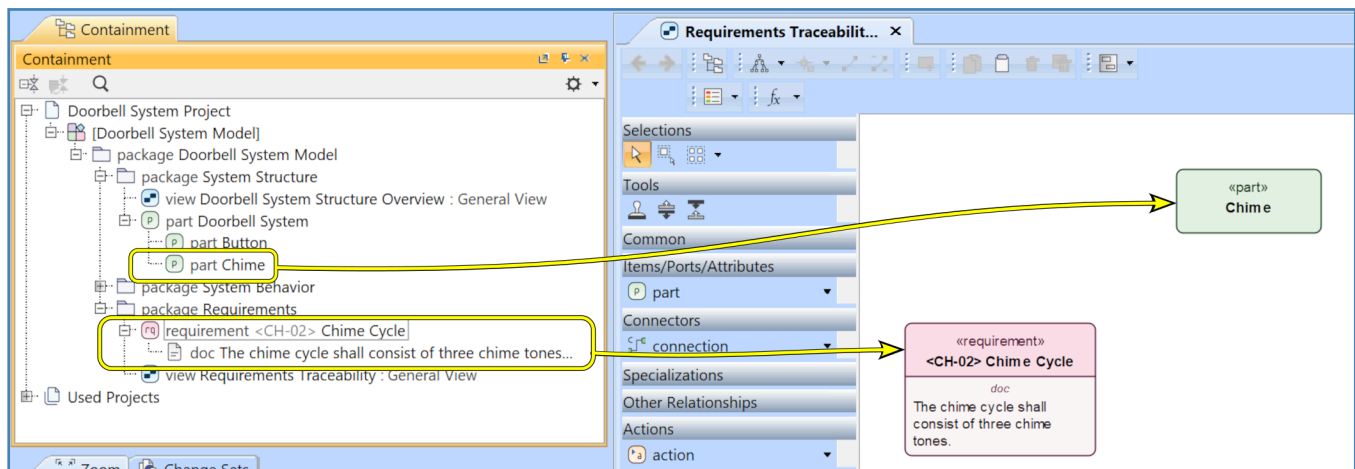


Figure 1-47 – Add the requirement and the chime part to the view

Drag the requirement and the Chime part from the containment tree to the view. Now, we are ready to think about modeling a relationship between the part and the requirement.

Importing Requirements

Entering requirements one-by-one this way would be very tedious in practice. In the long run, we expect *Cameo Systems Modeler* and other SysML V2 tools to implement features to import from and export to spreadsheets as well as features to connect to major requirements management tools. However, these features have not been fully developed and released yet.

Associate Parts and Requirements

Showing which system element satisfies which requirement is a key aspect of systems engineering. In the first version of SysML, there were a series of dashed line relationships that could be used to show various requirement related relationships such as:

- satisfaction of a requirement by an element
- derivation of a requirement from another requirement
- refinement of an element (including a requirement) by another element
- tracing an element (including a requirement) to another element

In SysML V2, these useful but syntactically fuzzy “dashed line” relationships have been replaced with more semantically precise constructs. Unfortunately, the focus of the team was on the text syntax and the option to show these relationships graphically was added to the language graphical syntax a bit late in the cycle. As such, the graphical approach is a bit of a work in progress and is not as simple as it was in SysML V1.

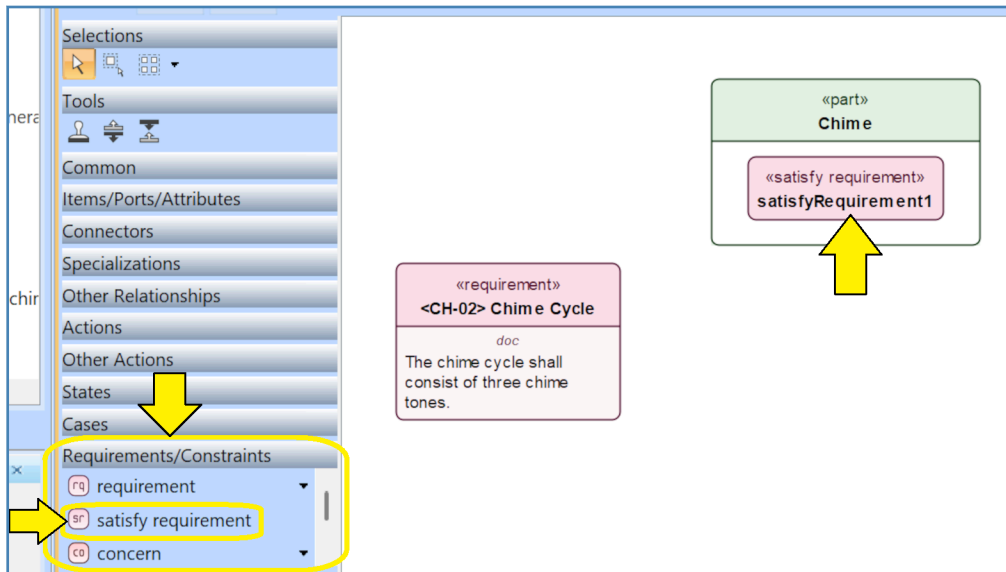


Figure 1-48 – Add a satisfy requirement element to the part

In the diagram palette, find the “Requirements/Constraints” section. Expand that section and find and select the “satisfy requirement” tool. Click inside the “Chime” part to create a new satisfy requirement element.

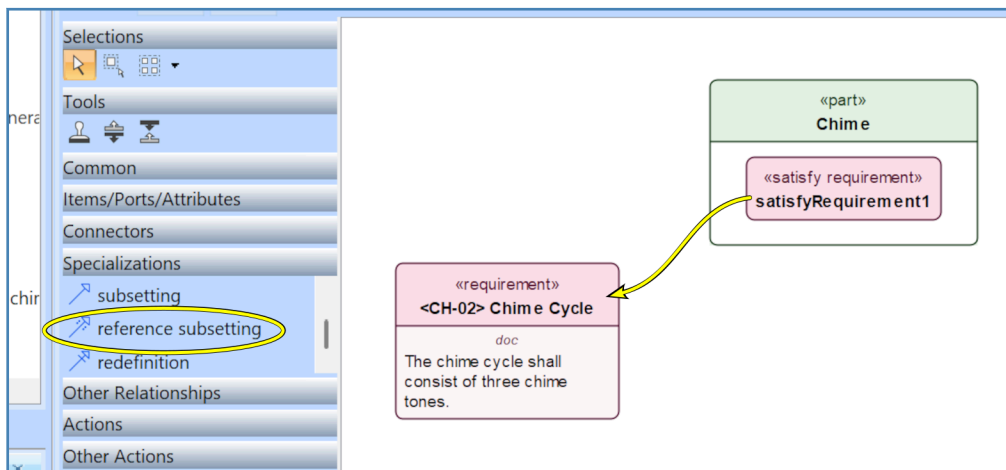


Figure 1-49 – Add a reference subsetting relationship

In the diagram palette, find the “Specializations” section. Expand that section and find and select the “reference subsetting” tool. Drag a relationship from the satisfy requirement element inside the part to the requirement element.

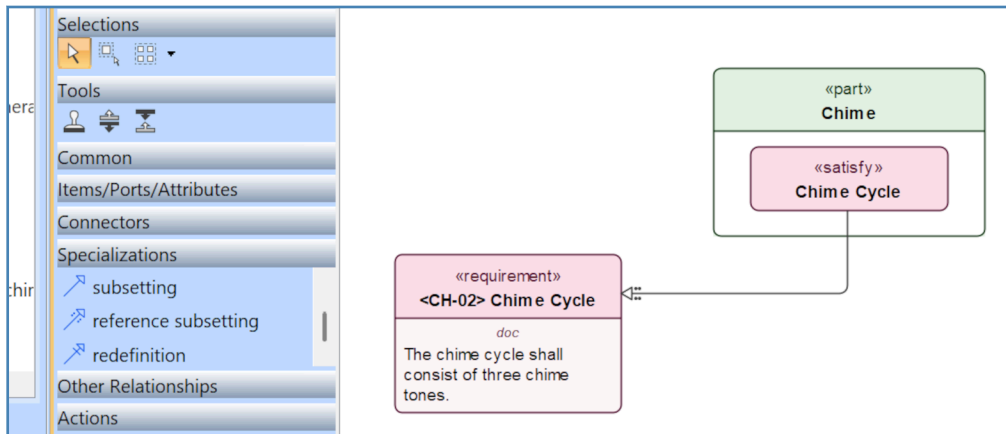


Figure 1-50 – Satisfy relationship complete

Notice that adding the reference subsetting relationship causes the satisfy element to change its name to match the name of the requirement.

We will be discussing requirements in greater detail late in the book.

Importing and Exporting Model Content

The tool provides very nice modeling project storage capability in the local repository. However, there are any number of reasons you might need to export or import either the modeling project or the SysML V2 text syntax. This section shows how to perform these four tasks.

Exporting the Modeling Project

First, we will look at exporting the modeling project. You can do this from the “File” menu.

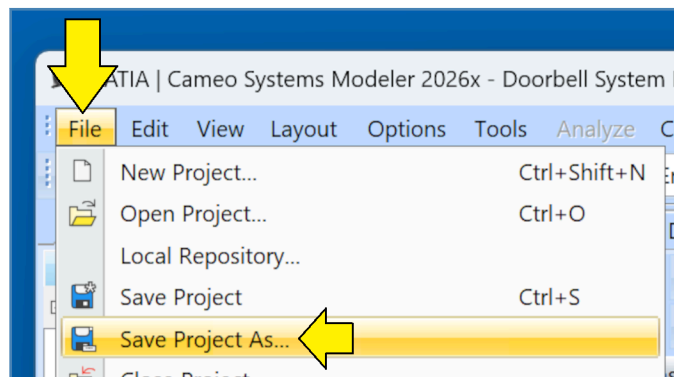


Figure 1-51 – Export the modeling project

With your modeling project loaded and open, pull down the “File” menu and select “Save Project As...”

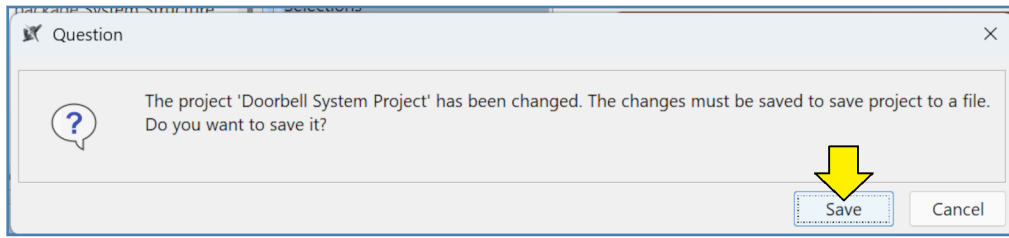


Figure 1-52 – Save the modeling project

The tool may warn you that you need to save the modeling project before continuing. In this case click “Save” to save the project.

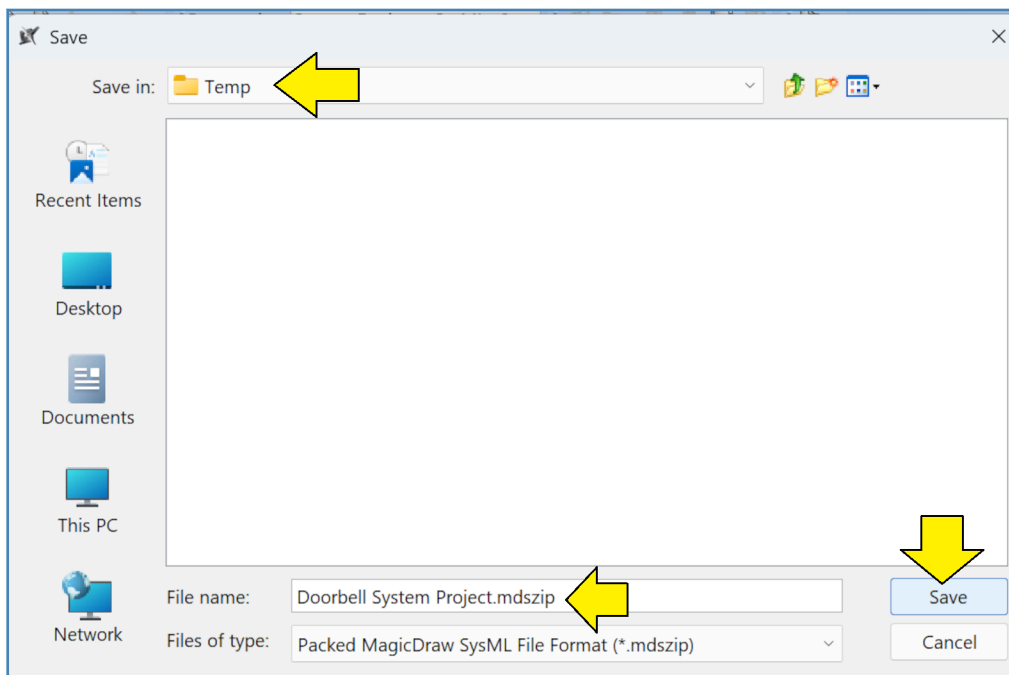


Figure 1-53 – Select location and name

- Navigate to the directory where you would like the exported modeling project saved.
- The tool will suggest a default name for the project. Adjust the name as needed.
- Click “Save”.

Exporting the Model SysML V2 Text Syntax

You can also export the SysML V2 text syntax from the “File” menu.

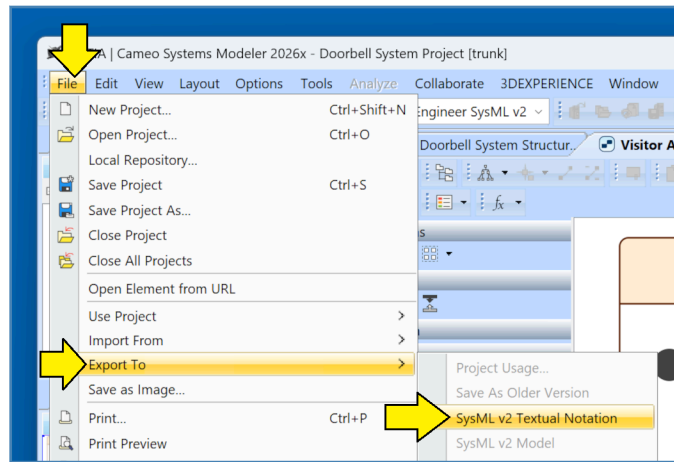


Figure 1-54 – Export the SysML V2 text syntax

With your modeling project loaded and open, pull down the “File” menu and select “Export To” and then “SysML V2 Textual Notation”.

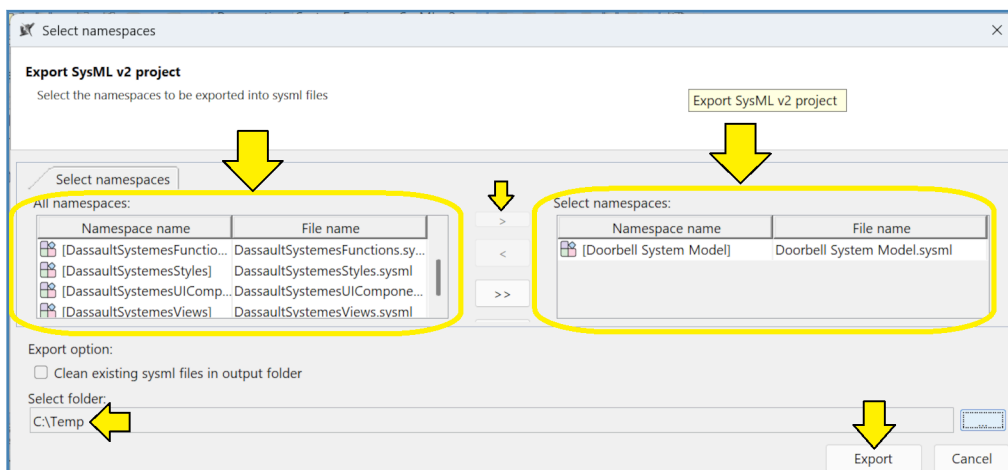


Figure 1-55 – Select namespace(s) and export

Next, you will have an opportunity to select namespaces to export. In most cases (like this example) we will be modeling with a single namespace and the tool will present that in the right-hand box. However, it is possible to have more than one namespace in a project. In this case, the right-hand box will show multiple namespaces, and multiple “*.sysml” files will be generated when you export them.

The modeling project also leverages a large number of standard libraries. These are shown in the left-hand box. If you want, some of these can be moved from the left-hand box to the right-hand box (or vice versa) using the arrows in the center.

When you are satisfied with the list of namespaces to export, select a location for the files and click “Export”.

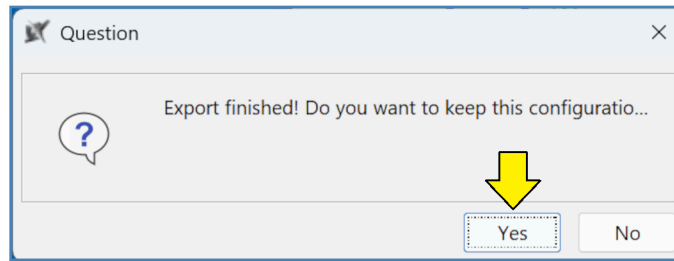


Figure 1-56 – Agree to keep configuration

If you are prompted to keep the configuration (whatever that means) click “Yes”.

Importing a Modeling Project

You may want to import a modeling project from another source such as one of the examples provided by this book. Modeling projects can be imported from the local repository

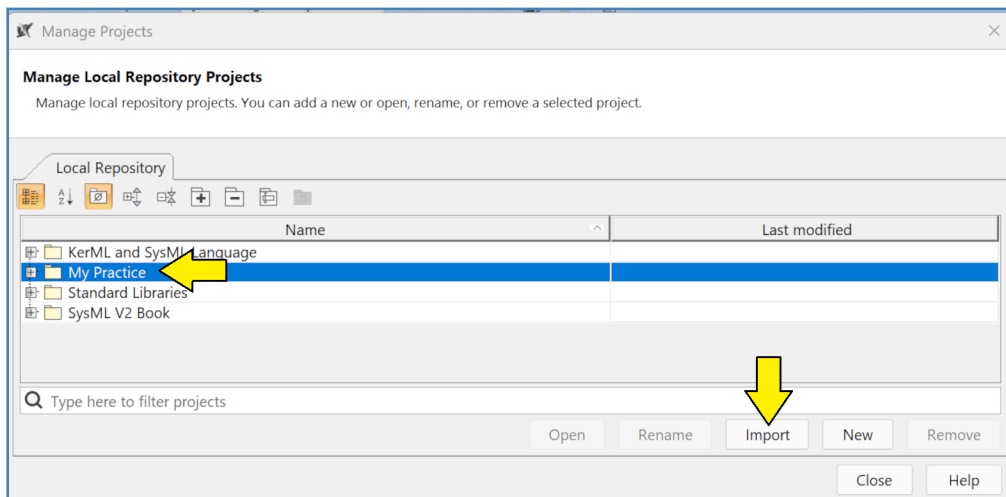


Figure 1-57 – Import a modeling project

From within the local repository, select a category and click “Import”.

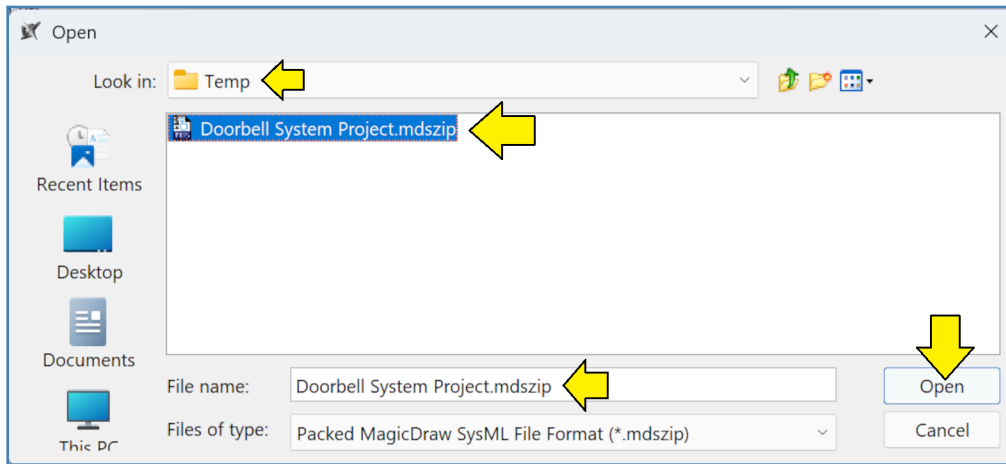


Figure 1-58 – Select the file to import

Navigate to the directory where the modeling project file is stored. Select the file. Click “Open”.

Note that under the surface, the tool assigns unique IDs to each project. These are stored in the modeling project file. Even if you rename the modeling project file, you may encounter warnings about having the same model open already.

Importing some SysML V2 Text Syntax

You can also import the SysML V2 text syntax from the “File” menu.

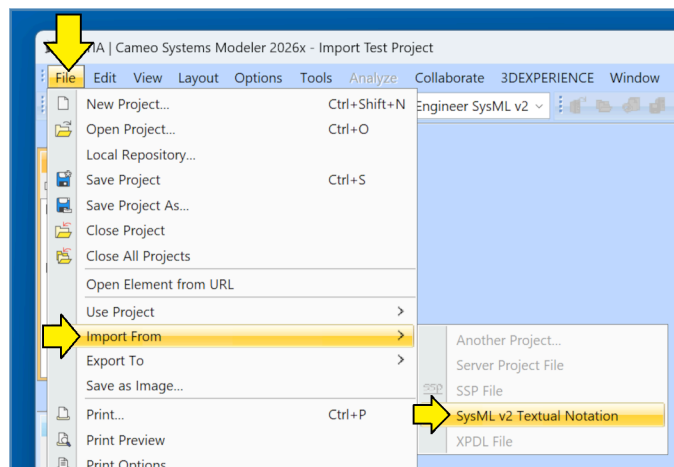


Figure 1-59 – Import some SysML V2 text syntax

With your modeling project loaded and open, pull down the “File” menu and select “Import From” and then “SysML V2 Textual Notation”.

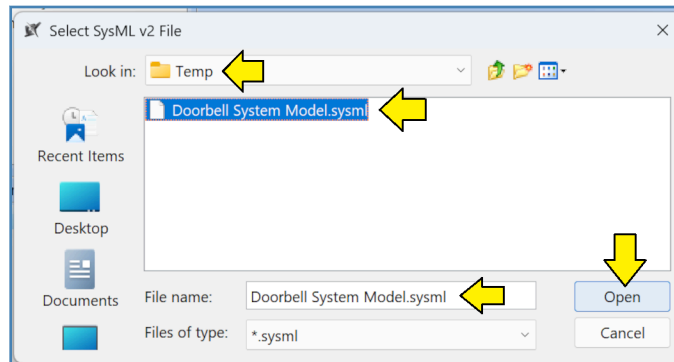


Figure 1-60 – Select file of SysML V2 text syntax to import

Navigate to the directory where the SysML V2 text syntax file is stored. Select the file. Click “Open”.

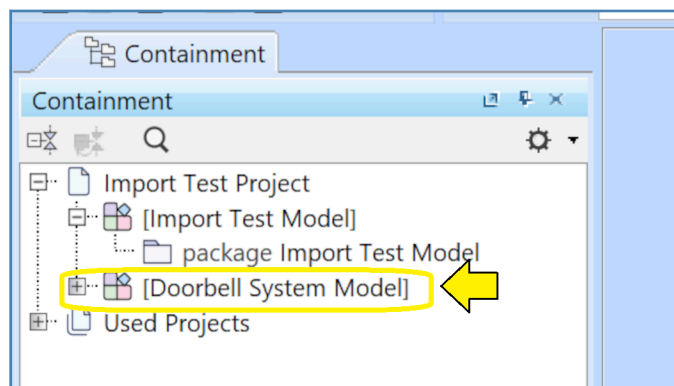


Figure 1-61 – Import Complete

The imported text syntax is now present in our modeling project containment tree. However, on closer inspection, we will notice a few things.

- 1) The tool has added the imported content as a new namespace in the project.
- 2) The model elements and relationships have been imported.
- 3) The views (which are also model elements) have also been imported, but....
- 4) The views are empty. This lack of content in the views reflects the current state of the SysML V2 standard. The syntax for declaring the contents of a view has not been fully worked out yet.